

Personalized Federated Hyperdimensional Computing via Shared-Personal Prototype Memory Decomposition

Anonymous Author

Abstract—Federated learning on edge devices requires models that are both efficient and robust to heterogeneous client data. Hyperdimensional computing (HDC) is a promising paradigm due to its lightweight, prototype-based representation and low computation and communication cost. However, existing federated HDC methods rely on a single shared model, which leads to poor alignment under client heterogeneity and limits personalization. The key challenge is that HDC encodes discriminative information in a fully distributed prototype memory, where shared and client-specific information are inherently entangled, making it unclear what should be federated.

We propose HoRU, a personalized federated HDC framework that addresses this challenge through low-rank prototype decomposition. HoRU separates each client's prototype memory into shared and client-specific components, enabling selective synchronization across clients while preserving local structure. It further employs class-adaptive updates to modulate aggregation strength based on local error signals, improving robustness under heterogeneous conditions. By communicating only the shareable component, HoRU maintains the efficiency advantages of HDC while enabling effective personalization.

Across seven federated benchmarks, HoRU improves mean accuracy by +4.3% points over HyperFeel and +21.2% points over FedHDC. It also achieves significant system-level gains, reducing client-round latency by $3.0\times$ and communication volume by $62.5\times$ compared to prior federated HDC approaches.

Index Terms—Personalized Federated Learning, Hyperdimensional Computing, HoRU

I. INTRODUCTION

The increasing use of edge devices in the Internet of Things, wearable systems, and cyber-physical environments has created demand for machine learning methods that can operate under tight resource constraints [1]. Traditional deep neural networks, while effective in many settings, often require levels of computation, memory, and energy that are difficult to support on such devices. Hyperdimensional Computing (HDC) [2], [3] has emerged as a practical alternative. HDC encodes data into high-dimensional vectors, referred to as *hypervectors*, and performs learning using simple arithmetic operations that are largely element-wise and independent, resulting in inherently lightweight and highly parallel computation. These properties enable lightweight, fast, and robust learning, making HDC well suited for deployment in resource-constrained edge environments.

While HDC provides an efficient way to train locally, an edge device usually observes only its own limited and biased data. In many real deployments, local samples are noisy, scarce, and strongly tied to one user, one context, or one sensing condition. Training only on-device therefore restricts generalization. At the same time, sending raw data to a central

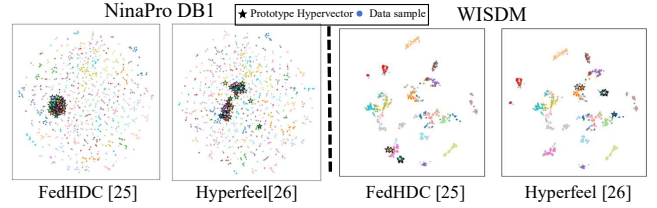


Fig. 1. Motivation for personalized federated HDC. Under natural client heterogeneity, a single globally shared prototype arrangement learned by existing methods [6], [12] can become misaligned with local sample geometry.

server increases communication cost and introduces latency while increasing privacy and security concerns.

Federated Learning (FL) addresses these challenges [4], [5] by allowing clients to collaborate through model exchange rather than centralized data collection. HDC also aligns with the requirements of edge-based FL, and recent work has begun to explore this combination. Early federated HDC studies [6]–[9] demonstrated several attractive properties such as fast local training, reduced communication overhead, and robustness to noise. However, these methods still largely adopt a *single global model* view. Under heterogeneous client distributions, aggregating a single model across all clients can suppress client-specific attributes, resulting in a one-size-fits-all model that is not well aligned with any individual client.

Fig. 1 demonstrates this mismatch. On NinaPro DB1 [10] and WISDM [11], we compare the class hypervectors produced by existing federated HDC methods [6], [12] with the corresponding client data distributions. Under heterogeneous conditions, the globally aggregated prototypes fail to align with local sample structure. As a result, the class hypervectors do not accurately represent client-specific data patterns, leading to reduced class separability at the client level.

Consequently, the primary challenge in federated HDC extends beyond communication efficiency: it lies in determining *what information should be shared across clients and what should remain client-specific*. This challenge directly motivates the need for *personalization*, i.e., enabling each client to maintain a model that is tailored to its local data while still benefiting from shared knowledge.

However, unlike neural network-based federated learning, where model components can be modularly adapted to support personalized models, HDC fundamentally lacks explicit structural separation. Instead, it maintains a *prototype memory* composed of per-class hypervectors, where information is encoded in a fully distributed manner across all dimensions. As a result, client-specific and globally shared information are inherently entangled within the same representation, making it difficult to

isolate which components can be safely aggregated. Therefore, achieving effective *personalized* federated learning in HDC is not a straightforward extension, but is fundamentally coupled with the underlying representation design.

To address this challenge, we propose HoRU (**H**yperdimensional **l**ow-**R**ank **U**pdate), a personalized federated learning framework for HDC that separates what should be shared from what should remain personal. Our key insight is that prototype hypervectors contain both cross-client regularities and client-dependent variations, which can be separated through a low-rank update structure.

Specifically, HoRU represents the globally shared information as a low-rank component that is synchronized across clients, while retaining a complementary local component to capture client-specific structure. During training, clients collaborate only through the shared low-rank updates, significantly reducing communication overhead, while preserving local adaptability through the unsynchronized residual memory. Furthermore, HoRU employs class-adaptive updates, where the strength of synchronization is modulated based on local error signals for selective alignment across clients: classes with higher local errors are updated more aggressively across clients, while well-learned classes are updated more conservatively.

By operating directly on prototype memory, HoRU provides a mechanism for personalization in HDC without sacrificing its efficiency advantages. Rather than relying on external model partitioning, it learns, *within the representation itself*, which information should be globally shared and which should remain client-specific. This design allows only the shareable component of memory to be communicated, significantly reducing communication cost.

This work makes the following key contributions:

- We propose HoRU, a personalized federated learning framework for hyperdimensional computing that decomposes class hypervectors into shared and client-specific components.
- We develop a low-rank learning and communication scheme that represents class hypervectors through shared and personal coefficients, communicates only the shared coefficients, and uses class-wise updates to improve personalization while reducing communication overhead.
- Across seven federated benchmarks, HoRU improves mean accuracy by +4.3% points over HyperFeel [12] and +21.2% points over FedHDC [6]. It also provides achieves system-level improvements, reducing client-round latency by $3\times$ and communication volume by $62.5\times$ compared to both baselines.

The remainder of this paper is organized as follows. Sec. II provides background on hyperdimensional computing and its use in federated learning under heterogeneous clients. Sec. III presents the proposed HoRU framework, including the shared–personal prototype decomposition, local learning, and server-side aggregation. Sec. IV evaluates HoRU across multiple federated benchmarks, analyzing both accuracy and system efficiency. Sec. V discusses related work, and Sec. VI concludes the paper.

II. BACKGROUND

A. Hyperdimensional Computing

HDC is a learning paradigm based on high-dimensional distributed representations, called *hypervectors*, together with simple algebraic operations in that space. Given an input sample x , an encoder maps it to a hypervector

$$h = \phi(x) \in \mathbb{R}^D,$$

where D is typically very large. The encoder is designed so that input samples can be represented in a common hypervector space, while preserving useful latent structure from the original input, making it possible to learn and compare samples directly through operations on their hypervectors. Learning in HDC is then performed by constructing *class hypervectors*, which serve as the classifier in hypervectorspace [13], [14]. In the simplest case, encoded samples are grouped by class and averaged, so that each class is represented by one class hypervector [15], [16]. A common refinement is error-driven learning: when a sample is misclassified, the encoded hypervector is element-wise added to the true-class hypervector and element-wise subtracted from the wrongly predicted class hypervector. Prediction is made by comparing the encoded query hypervector with the stored class hypervectors using a similarity measure such as cosine similarity or dot product [17].

B. Hyperdimensional Computing in FL under Heterogeneity

The prototype-based learning of HDC fits federated learning (FL), where multiple clients train collaboratively while keeping raw data local [18]–[20]. HDC performs learning in hypervector space through simple operations such as element-wise addition, element-wise subtraction, and similarity-based prediction, instead of repeated backpropagation through a deep network. This lightweight computation supports fast local updates on edge, IoT, and robotic devices, where compute, memory, and communication are often limited [21], [22]. In federated HDC, each client constructs class hypervectors from its local data, sends compact model-side information to the server, and receives an aggregated update in return [7], [9]. Prior work shows that this representation can reduce local training overhead and communication cost in edge-oriented federated settings [12], [23]. These properties make HDC a practical foundation for FL under heterogeneous client conditions.

III. PROPOSED METHOD: HoRU

A. Problem Setup and Notation

We consider personalized federated learning in hyperdimensional computing (HDC) with N clients, K classes, and hypervector dimension D . Client i owns a local dataset

$$\mathcal{D}_i = \{(x_{i,j}, y_{i,j})\}_{j=1}^{|\mathcal{D}_i|},$$

where $x_{i,j}$ is an input sample and $y_{i,j} \in \{1, \dots, K\}$ is its class label. A shared encoder $\phi(\cdot)$ maps each input to a hypervector

$$h_{i,j} = \phi(x_{i,j}) \in \mathbb{R}^D.$$

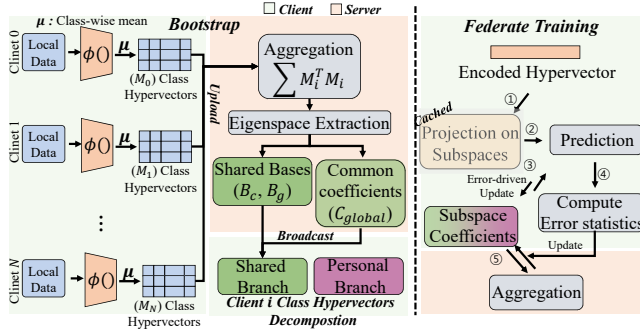


Fig. 2. Overview of HoRU. The method first builds a decomposition of the class hypervectors during the bootstrap stage, and then refines only the shared branch during federated training.

Let $n_{i,k}$ denote the number of local training samples of class k on client i . Using these encoded samples, each client constructs a matrix of class hypervectors

$$M_i \in \mathbb{R}^{K \times D},$$

whose k -th row is the class hypervector of class k on client i . We first compute the class-wise mean hypervector

$$\widetilde{M}_i[k] = \frac{\sum_{(x,y) \in \mathcal{D}_i} \mathbb{I}[y=k] \phi(x)}{\max(n_{i,k}, 1)}, \quad (1)$$

and then row-normalize it:

$$M_i[k] = \begin{cases} \text{row_normalize}(\widetilde{M}_i[k]), & n_{i,k} > 0, \\ \mathbf{0}, & n_{i,k} = 0. \end{cases} \quad (2)$$

Thus, M_i is the set of class hypervectors induced by the local data of client i . In personalized federated learning, the key question is not whether class hypervectors should be fully shared or fully local, but which part should be shared across clients and which part should remain client-specific.

The goal of HoRU is therefore to decompose each client's matrix of class hypervectors into a *shared branch* and a *personal branch*, and to synchronize only the shareable part. Here, a *branch* refers to a set of hypervectors together with their associated coefficients in the prototype representation. In our design, the shared branch is further divided into a *common branch* and a *global branch*. The common branch captures shared class hypervector structure that each client can further adapt locally, while the global branch preserves the server-shared class hypervector structure as a stable reference during federated training. This separation allows local adjustment in the common branch without directly overwriting the global branch. The personal branch preserves client-specific class hypervector structure that remains local.

B. Overview

Fig. 2 illustrates the learning procedure of HoRU. HoRU decomposes the class hypervectors into three representations: a common branch, a global branch, and a personal branch. The common and global branches together form the *shared branch*, while the personal branch remains local to each client.

The method performs two phases: the *bootstrap stage* and *federated training stage*. The purpose of the *bootstrap stage*

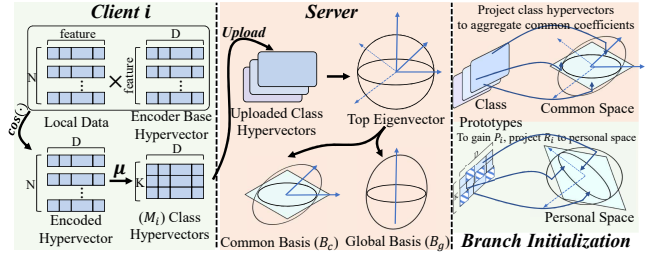


Fig. 3. Bootstrap stage of the shared-personal class hypervector decomposition. The server computes the common and global bases from client class hypervectors, and each client initializes the shared and personal branches accordingly.

is to initialize this decomposition before iterative federated updates begin. Each client first encodes its local data and constructs its matrix of class hypervectors M_i . The server then uses the collection of client class hypervectors to compute a *common basis* and a *global basis*. Using the common basis, the server projects the client class hypervectors, averages the resulting common coefficients across clients, and forms the initial common shared state. Based on the remaining client-specific component, each client initializes its own personal branch. After this stage, each client has a decomposition consisting of a shared branch (common + global) and a personal branch.

The purpose of the *federated training stage* is to refine the shared branch while preserving the personal branch. In each round, an encoded hypervector is represented by *common coefficients*, *global coefficients*, and *personal coefficients* under the learned branch bases (in a form of hypervectors), and prediction is performed directly from these coefficients with cosine similarity. When a sample is misclassified, the client applies an error-driven update to its local coefficients.

After local training, each client computes *class-wise error statistics* on its local data and converts them into *class-wise update weights*. The client then uploads the shared coefficients, i.e., the common and global coefficients, while the personal coefficients remain local. This uplink is the only information transmitted to the server for synchronization.

At the server, the uploaded shared coefficients are combined across clients by class-wise weighted summation using the class sample counts, updating shared coefficients. The server then broadcasts these aggregated shared coefficients. After receiving them, each client updates its local shared branch using its local class-wise update weights. In this way, classes with higher local error follow the aggregated shared coefficients more strongly, while classes that are already well adapted preserve more of their local class hypervectors.

C. Extracting Shared Bases

The goal of the *bootstrap stage* is to extract the shared structure that is consistent across clients. Since each client's class hypervectors contain both shared and client-specific variations, we construct a global representation by aggregating second-order statistics across clients. Intuitively, this step identifies directions in hypervector space that capture common patterns, which will later form the shared branch of the model.

To initialize the shared personal brach decomposition, each client first encodes its local samples into hypervectors using a shared HDC encoder initialized by the server. Following [24], each encoder base hypervector is sampled from a standard normal distribution. Let $\{b_\ell\}_{\ell=1}^D$ denote the encoder base hypervectors. For a local sample $x_{i,j} \in \mathbb{R}^F$, the encoded hypervector is

$$h_{i,j} = \phi(x_{i,j}), \quad [h_{i,j}]_\ell = \cos(b_\ell^\top x_{i,j}), \quad \ell = 1, \dots, D. \quad (3)$$

For client i , the encoded hypervectors are grouped by class and averaged to form class hypervectors. Stacking them gives the client class hypervectors

$$M_i \in \mathbb{R}^{K \times D}.$$

The server then uses the collection of client class hypervectors $\{M_i\}_{i=1}^N$ to compute the shared branch, which consists of a common branch and a global branch. Formally, the server computes the pooled covariance

$$\Sigma = \sum_{i=1}^N M_i^\top M_i. \quad (4)$$

It then applies eigen decomposition to obtain

$$B = [B_c, B_g] \in \mathbb{R}^{D \times (r_c + r_g)},$$

where $B_c \in \mathbb{R}^{D \times r_c}$ is the common basis and $B_g \in \mathbb{R}^{D \times r_g}$ is the global basis. The common basis and global basis together define the shared branch.

D. Initialization of the Shared and Personal Branches

After obtaining the shared bases, we initialize the shared and personal branches by decomposing each client's prototype memory into shared and residual components. The key idea is to project client prototypes onto the shared subspace to extract transferable information, while assigning the remaining residual to the personal branch. This initialization establishes a clear separation between shared and client-specific structure before federated training begins.

Using the bootstrapped shared bases, the server initializes the shared branch and each client initializes its personal branch. To this end, the server first projects class hypervectors of each client onto the common basis:

$$C_i^{\text{tot}} = M_i B_c. \quad (5)$$

It then computes the class-wise common consensus

$$C_{\text{global}}[k] = \frac{\sum_{i=1}^N n_{i,k} C_i^{\text{tot}}[k]}{\sum_{i=1}^N n_{i,k}}, \quad (6)$$

where $n_{i,k}$ is the number of local samples of class k at client i . If $\sum_{i=1}^N n_{i,k} = 0$, then $C_{\text{global}}[k]$ is set to $\mathbf{0}$.

In the bootstrap stage, only the common branch is initialized from the shared consensus. The global branch and the client-specific common correction are initialized to zero:

$$C_i^{(0)} = C_{\text{global}}, \quad G_i^{(0)} = \mathbf{0}, \quad \Delta_i^{(0)} = \mathbf{0}. \quad (7)$$

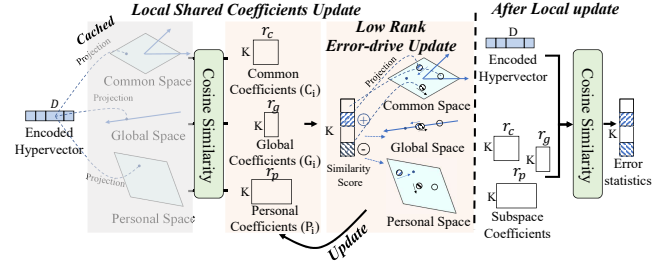


Fig. 4. Local coefficient learning of class hypervectors in the proposed framework. Each encoded sample is projected onto the learned common, global, and personal bases, compared with the stored class hypervector coefficients, and used to update the shared and personal branches.

After receiving the broadcast shared state, each client initializes its personal branch from the residual part of its local class hypervectors after removing the shared subspace. Let

$$B = [B_c, B_g], \quad R_i = M_i(I - BB^\top).$$

Client i then computes its personal basis $B_p \in \mathbb{R}^{D \times r_p}$ as the top- r_p right singular vectors of R_i . The initial personal coefficients are

$$P_i^{(0)} = R_i B_p. \quad (8)$$

The resulting initialized class hypervector matrix is

$$M_i^{(0)} = \text{row_normalize} \left(\underbrace{C_{\text{global}} B_c^\top + G_i^{(0)} B_g^\top}_{\text{shared}} + \underbrace{\Delta_i^{(0)} B_c^\top + P_i^{(0)} B_p^\top}_{\text{personal}} \right). \quad (9)$$

E. Local Coefficients Learning with HDC

After the bootstrap stage, client i has a shared branch and a personal branch for its class hypervectors. The purpose of local learning is to refine these class hypervectors directly in class hypervectors subspace, so that training can be performed without repeatedly reconstructing a full D -dimensional class hypervector matrix. This design enables efficient learning while preserving the decomposition structure, allowing shared and personal components to be updated in a coordinated manner.

For an encoded sample $h = \phi(x)$, the client projects h once onto the common, global, and personal basis and caches the resulting coordinates:

$$z_c = h B_c, \quad z_g = h B_g, \quad z_p = h B_p. \quad (10)$$

For class k , the shared branch stores common coefficients $C_i[k]$ and global coefficients $G_i[k]$. The personal branch stores a client-specific correction $\Delta_i[k]$ in the common basis and personal coefficients $P_i[k]$ in the personal basis. Because $C_i[k]$ and $\Delta_i[k]$ lie in the same common basis, they combine additively. The resulting class hypervector coordinates are

$$s_{i,k}^{\text{full}} = [C_i[k] + \Delta_i[k], G_i[k], P_i[k]], \quad (11)$$

and the corresponding query coordinates are

$$q_i^{\text{full}}(h) = [z_c, z_g, z_p]. \quad (12)$$

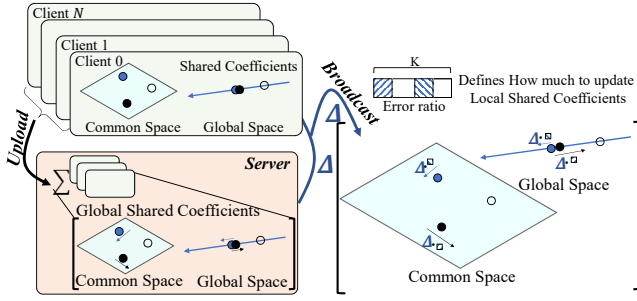


Fig. 5. Server-side shared-coefficient aggregation and local shared-branch update. Clients upload only the common and global coefficients, the server aggregates them class-wise, and each client then uses its local class-wise update weights to determine how strongly to follow the aggregated shared update.

Prediction is then performed by cosine similarity directly with coefficients:

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} \cos(q_i^{\text{full}}(h), s_{i,k}^{\text{full}}). \quad (13)$$

If $\hat{y} \neq y$, the client updates the corresponding coefficients rows by a push-pull rule. The shared coefficients are updated with learning rate η_s :

$$C_i[y] \leftarrow C_i[y] + \eta_s z_c, \quad C_i[\hat{y}] \leftarrow C_i[\hat{y}] - \eta_s z_c, \quad (14)$$

$$G_i[y] \leftarrow G_i[y] + \eta_s z_g, \quad G_i[\hat{y}] \leftarrow G_i[\hat{y}] - \eta_s z_g, \quad (15)$$

and the personal coefficients are updated with learning rate η_p :

$$\Delta_i[y] \leftarrow \Delta_i[y] + \eta_p z_c, \quad \Delta_i[\hat{y}] \leftarrow \Delta_i[\hat{y}] - \eta_p z_c, \quad (16)$$

$$P_i[y] \leftarrow P_i[y] + \eta_p z_p, \quad P_i[\hat{y}] \leftarrow P_i[\hat{y}] - \eta_p z_p. \quad (17)$$

This update rule gives each branch a distinct role. The common coefficients C_i and global coefficients G_i refine the shared branch, while the client-specific correction Δ_i and the personal coefficients P_i refine the personal branch. Because both C_i and Δ_i are defined in the common basis, the common basis carries both shared structure and client-specific adaptation. This gives the common basis a larger functional role than the global basis during local learning. The global coefficients G_i remain in the shared branch and are later synchronized through server-side aggregation, whereas Δ_i provides a client-specific correction to the common coefficients using local data.

After local training, client i computes class-wise error statistics from its final class hypervector coefficients. Let $t_{i,k}$ be the number of local training samples of class k , and let $w_{i,k}$ be the number of those samples that remain misclassified after local training. The class-wise error ratio is then

$$e_{i,k} = \frac{w_{i,k}}{\max(t_{i,k}, 1)}. \quad (18)$$

This class-wise error statistic remains local to client i and is later used to determine the class-wise update weight for incorporating the aggregated shared coefficients after server-side aggregation.

F. Server-Side Shared-Coefficient Aggregation and Local Shared-Branch Update

After local updates, the shared components must be aggregated across clients while respecting client-specific differences. The key idea is to perform class-wise aggregation of shared coefficients, and then allow each client to adapt the global update based on its local error. This ensures that classes requiring more correction receive stronger updates, while well-learned classes remain stable, enabling selective and adaptive synchronization.

Concretely, post-local training, client i uploads only its shared coefficients, namely the common coefficients C_i and the global coefficients G_i . The personal coefficients (Δ_i, P_i) remain local. Using the uploaded shared coefficients, the server computes a class-wise shared consensus:

$$\bar{C}[k] = \frac{\sum_{i=1}^N n_{i,k} C_i[k]}{\sum_{i=1}^N n_{i,k}}, \quad \bar{G}[k] = \frac{\sum_{i=1}^N n_{i,k} G_i[k]}{\sum_{i=1}^N n_{i,k}}, \quad (19)$$

where $n_{i,k}$ is the number of local samples of class k at client i . If $\sum_{i=1}^N n_{i,k} = 0$, then $\bar{C}[k]$ and $\bar{G}[k]$ are set to $\mathbf{0}$. Because the aggregation is weighted class-wise by $n_{i,k}$, a client with no local samples for class k contributes nothing to row k .

Each client next updates its own shared branch using its local class-wise error statistics. If class k still has high local error, the client should follow the aggregated shared coefficients more strongly; if class k is already modeled well locally, the client should change less. We therefore define the local class-wise update weight as

$$\omega_{i,k} = \begin{cases} e_{i,k}, & t_{i,k} > 0, \\ 0, & t_{i,k} = 0, \end{cases} \quad (20)$$

where $e_{i,k}$ is the class-wise error ratio defined above.

Let

$$U_i = [C_i \ G_i], \quad \bar{U} = [\bar{C} \ \bar{G}].$$

Client i then updates the shared coefficients row-wise as

$$U_i^+[k, :] = U_i[k, :] + \eta_g \omega_{i,k} (\bar{U}[k, :] - U_i[k, :]). \quad (21)$$

G. Inference

Inference follows the same decomposition principle used during training. Instead of reconstructing full prototype hypervectors, predictions are made directly in the coefficient space by combining shared and personal components. This preserves efficiency while ensuring that both globally shared structure and client-specific adaptations contribute to the final decision.

Inference uses the same coefficient representation of the class hypervectors as local training. Given an input sample x , client i first encodes it into a query hypervector $x_{hv} = \phi(x)$. The query hypervector is then projected onto the common, global, and personal basis:

$$z_c = x_{hv} B_c, \quad z_g = x_{hv} B_g, \quad z_p = x_{hv} B_p. \quad (22)$$

These three projections give the query coefficients

$$q_i(x) = [z_c, z_g, z_p]. \quad (23)$$

For class k , client i stores the class hypervector through its branch coefficients. The corresponding class hypervector coefficients are

$$u_{i,k} = [C_i[k] + \Delta_i[k], G_i[k], P_i[k]]. \quad (24)$$

Here, $C_i[k] + \Delta_i[k]$ gives the common-branch coefficients with client-specific correction, $G_i[k]$ gives the global-branch coefficients, and $P_i[k]$ gives the personal-branch coefficients.

Thus, inference compares the query with each class hypervector through the shared and personal branches together. The prediction is made directly from the branch coefficients, without explicitly reconstructing a full D -dimensional class hypervector.

H. Communication and Complexity

A key advantage of HoRU is that communication and computation scale with the low-rank representation rather than the full hypervector dimension. By transmitting only shared coefficients and keeping personal components local, the method significantly reduces communication overhead while maintaining model expressiveness. This section quantifies these efficiency gains.

A full class hypervector stores one D -dimensional class hypervector for each of the K classes, so it contains $K \times D$ values. In HoRU, each client uploads only the common coefficients C_i and global coefficients G_i , which together contain $K \times (r_c + r_g)$ values per round. The client-specific common correction Δ_i , the personal coefficients P_i , the personal basis $B_{p,i}$, and the class-wise error statistics used to form local class-wise update weights remain on the client. Ignoring negligible protocol metadata, the recurring communication ratio relative to uploading a full class hypervector memory is therefore

$$\frac{\text{payload}_{\text{ours}}}{\text{payload}_{\text{full}}} \approx \frac{r_c + r_g}{D}. \quad (25)$$

Because $r_c + r_g \ll D$, the recurring communication cost is substantially reduced.

The server-side shared-coefficient aggregation step is also low-rank. The server receives only the uploaded common and global coefficients, computes class-wise weighted averages, and broadcasts the aggregated shared coefficients back to the clients. Each client then updates its local shared branch by moving its common and global coefficients toward the broadcast shared coefficients, with a separate local class-wise update weight controlling how strongly each class follows the shared update.

Computation likewise remains in coefficient space. For an encoded hypervector h , projection onto the common, global, and personal bases costs $O(D(r_c + r_g + r_p))$. Once projected, class scoring is performed in the concatenated coefficient space, so comparing against all K classes costs $O(K(r_c + r_g + r_p))$. This avoids repeated comparison against a full D -dimensional class hypervector memory.

The only additional one-time cost relative to a purely shared branch appears in the bootstrap stage, where each client computes its personal basis from the residual class hypervectors. Let $R_i = M_i(I - BB^\top) \in \mathbb{R}^{K \times D}$ and $B = [B_c, B_g]$.

Obtaining the personal basis $B_{p,i}$ requires a truncated rank- r_p decomposition only once at initialization. The client-side memory for this step is dominated by storing the class hypervectors and basis matrices, namely $O(KD + D(r_c + r_g + r_p))$ values.

Overall, the one-time bootstrap stage computes the decomposition, whereas the recurring federated rounds communicate and update only the shared coefficients. The resulting round cost therefore scales with the retained low-rank dimensions rather than with the full-dimension D .

IV. EXPERIMENTS

A. Experimental Setup

We evaluate on seven federated benchmarks: UCI-HAR, ISOLET, FEMNIST, WISDM, Synthetic, and NinaPro DB1. We use the natural user- or subject-level federated split whenever available; otherwise, we use the standard heterogeneous partition. Reported results are mean accuracies over multiple runs.

UCI-HAR [25] uses a subject-based split with 30 clients, ℓ_2 normalization, and a 30% test split when required by the benchmark pipeline. ISOLET [26] uses its original federated split with 8 clients, ℓ_2 normalization, and a 30% test split. FEMNIST [27] follows the original LEAF writer split, using the first 200 clients and the canonical train/test partition; inputs are ℓ_2 -normalized. WISDM [11] uses the smartphone-accelerometer modality with a user split over 51 clients, standardized features, a 30% test split, and the same per-client caps of 5,000 training and 1,000 test samples. Synthetic [27] uses the LEAF-style user split with 30 clients and no additional normalization. NinaPro DB1 [10] uses a subject split with the electromyography-and-glove modality over 27 clients, standardized features, a 30% test split, and per-client caps of 5,000 training and 1,000 test samples.

Baselines. We compare against HD baselines (**FedHDC** [6] and **HyperFeel** [12]) and neural federated baselines (**FedAvg** [5] and **DFL** [28]). These baselines cover prior HD-based federation as well as neural network-based FL techniques, i.e., global FL and state-of-the-art disentangled personalization.

Training protocol. All methods are trained for 25 communication rounds with full client participation, 3 local epochs per round, and batch size 32. For large-scale datasets (WISDM, NinaPro DB1), when the total number of training samples exceeds 100,000, we reduce the total to 50,000 through class-stratified subsampling within clients to keep the training budget comparable across datasets.

Neural network baselines. On FEMNIST, FedAvg use a CNN with two convolutional blocks (32 and 64 channels), ReLU, max-pooling, a 2048-unit fully connected layer, and a linear classifier. DFL uses the corresponding two-branch CNN. These models are trained with SGD with learning rate 0.06, without momentum, weight decay, or dropout; for DFL, the alignment and disentanglement weights are 1.0 and 0.1, respectively, with sample-wise standardization.

On the other datasets, the neural network baselines use a 4-layer MLP with hidden width 256 and no dropout, while

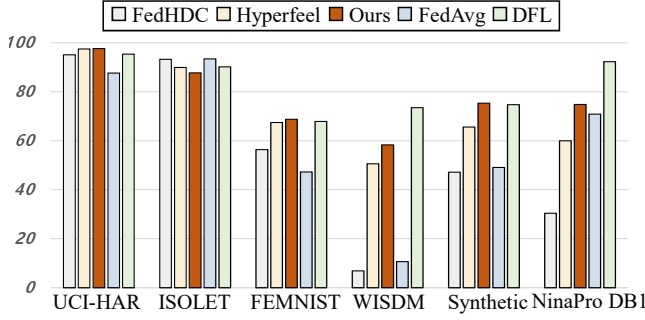


Fig. 6. Accuracy comparison across seven federated benchmarks.

DFL uses the corresponding two-branch MLP. These models are trained with Adam with learning rate 0.001. Personalized neural network baselines use one personalization step per round.

HD setting. For all HD-based methods, we use HD dimension $D = 2000$, HD learning rate 0.035, and no validation split. Our method uses a shared rank of 32, decomposed into common rank 24 and global-only rank 8, together with a personal rank of 64. Because no single rank configuration is uniformly optimal across all datasets, we additionally report a separate rank-screening study with dataset-specific alternatives.

Unless otherwise noted in the experiment-specific description, all experiments follow the training protocol, neural network baseline setting, and HD setting described above.

B. Accuracy

Fig. 6 summarizes the main accuracy comparison across seven federated benchmarks. Overall, HoRU improves over the prior HD-based baselines on most datasets. Using the mean accuracy across all seven benchmarks as a coarse summary, HoRU reaches 72.66%, compared with 68.28% for **HyperFeel** and 51.38% for **FedHDC**. This corresponds to gains of +4.38% and +21.28% percentage points, respectively.

We observe that HoRU is most effective on datasets where a single global model is not sufficient. On benchmarks such as WISDM and Synthetic, **FedAvg** yields significantly lower accuracy than personalized neural network baselines, which indicates a strong mismatch between global averaging and client-local structure. In contrast, HoRU substantially improves learning quality over global summation-based HD federation on these benchmarks, suggesting that separating shared and client-specific class hypervectors is more effective than treating the class hypervectors as one globally averaged object.

FEMNIST is particularly notable because the gap between HD and neural network-based methods becomes small. Here, HoRU reaches 68.78%, slightly above DFL at 67.83% and HyperFeel at 67.42%. This suggests that, even in a difficult many-class federated setting, a lightweight HD classifier can remain competitive when personalization is built into the class hypervector structure.

The results also show that HoRU achieves competitive learning performance compared to neural network-based baselines across most benchmarks, with the exception of NinaPro DB1. This difference reflects a fundamental trade-off between model

TABLE I
BOOTSTRAP COST BREAKDOWN OF HORU FOR SHARED-BASIS AND CLIENT-PERSONAL INITIALIZATION.

Scope	Component	Time (ms)
Server	Common/global basis computation	47.74
Server	Projection of client hypervectors	2.46
Server	Common/global coefficients build	0.37
Server	Server bootstrap total	50.57
Client	Residual class hypervectors build	0.11
Client	Personal basis computation	6.07
Client	Projection of Residual class hypervectors	0.08
Client	Cached common/global/personal query coefficient build	2.72
Client	Client bootstrap wall time	8.98

TABLE II
PER-ROUND CLIENT AND SERVER COST BREAKDOWN OF HORU AFTER BOOTSTRAP CACHING.

Scope	Component	Time (ms)
Client	Local similarity	0.40
Client	Local coefficients update	0.78
Client	Final train dataset prediction	0.42
Client	Class-wise error statistics	0.03
Client	Client round total	1.63
Server	Aggregation of common coefficients	0.19
Server	Aggregation of global coefficients	0.16
Client	client-side shared branch update with class-wise update weights	0.04
Mix	synchronization step total	0.39

capacity and efficiency. Neural network models typically rely on a large number of parameters to achieve high representational capacity, but this comes at the cost of substantial communication overhead in federated settings. In contrast, HDC-based methods, including HoRU, operate with compact prototype representations whose dimensionality directly determines both model capacity and computation/communication cost. In the following subsections, we discuss how HoRU leverages the efficiency of HDC representations to achieve significantly lower communication overhead (Sec. IV-C) and how to mitigate the performance gap due to the model capacity limitation (Sec. IV-D).

C. Efficiency

For the efficiency study, we design a controlled synthetic benchmark that enables fair comparison of round-level system cost across methods. The setup standardizes both dataset conditions and model scale, allowing us to isolate computational and communication overhead without confounding factors. The benchmark consists of 50 clients and 50 classes, with 1000 training samples per client and batch size 32 for one local epoch. To ensure consistent learning dynamics, each client is initialized with 500 misclassified samples. All experiments are conducted on CPU using an Intel(R) Core(TM) i9-14900KF. HDC-based methods use HD dimension $D = 2000$, with HoRU configured using common rank 24, global rank 8, and personal rank 64. For neural network baselines, FedAvg/CNN and DFL/CNN use a dummy CNN with hidden width $h = 16$, selected to match the model size of the HDC setting. For $K = 50$, $D = 2000$, $r_c = 24$, and $r_g = 8$, the bootstrap uplink is $4KD = 400.0$ KB, the bootstrap downlink is $4D(r_c + r_g) + 4Kr_c = 260.8$ KB, and the recurring uplink for common and global coefficients is $4K(r_c + r_g) = 6.4$ KB.

Table I reports the bootstrap cost of HoRU. This stage computes the common and global bases on the server and

TABLE III
FEDERATED-ROUND COMPUTE AND PAYLOAD ON THE REDUCED
SYNTHETIC BENCHMARK.

Method	Local round ms	Similarity ms	Update ms	Server step ms	Uploaded payload KB
HoRU (Ours)	1.63	0.40	0.78	0.39	6.4
FedHDC	4.72	2.52	2.20	1.25	400.0
HyperFeel	4.87	2.50	2.37	1.17	400.0
FedAvg CNN	553.85	–	553.85	1.73	412.5
DFL CNN	2063.61	–	2063.61	1.63	409.1

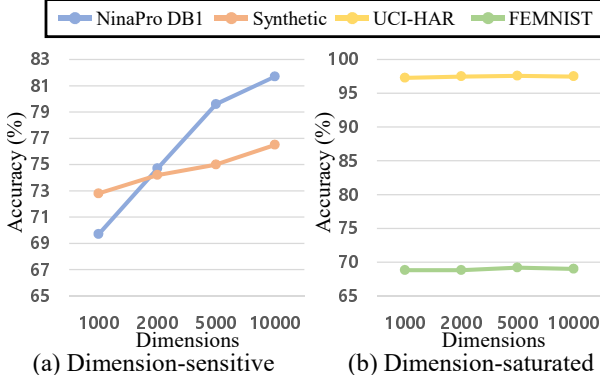


Fig. 7. Effect of HD dimension on mean personalized accuracy.

the personal basis on each client. The server bootstrap time is 50.57 ms, of which 47.74 ms is spent on common/global basis computation. The client bootstrap wall time is 8.98 ms, including 6.07 ms for personal basis computation and 2.72 ms for cached common/global/personal query coefficient construction. We report this stage separately because it occurs once before federated rounds begin.

Table II reports the recurring cost after bootstrap. Each client round takes 1.63 ms, including 0.40 ms for local similarity, 0.78 ms for local coefficient update, 0.42 ms for final full-train prediction, and 0.03 ms for class-wise error statistics. The server step takes 0.39 ms, including aggregation of common coefficients, aggregation of global coefficients, and server-side aggregation of class-wise update weights.

Table III compares the recurring round cost across methods. HoRU uses 1.63 ms per client round, compared with 4.72 ms for FedHDC and 4.87 ms for HyperFeel. FedAvg/CNN uses 553.85 ms, and DFL/CNN uses 2063.61 ms. The neural network baselines spend more time in local training because they rely on backpropagation, and DFL additionally maintains two branches during local optimization.

It also shows the recurring upload payload. HoRU uploads 6.4 KB per round, while FedHDC and HyperFeel upload 400.0 KB, FedAvg/CNN uploads 412.5 KB, and DFL/CNN uploads 409.1 KB. In our method, clients upload only the common and global coefficients, while the personal branch remains local.

These results separate the one-time bootstrap stage from the recurring federated round. In HoRU, the bootstrap stage adds an initial cost for basis construction, while the recurring round uses lower computation and lower upload payload.

UCI-HAR	-0.6	-0.4	-2.4	-2.9	-1.3
ISOLET	-0.1	-0.3	1.0	0.6	-2.5
FEMNIST	-14.4	0.5	-3.3	-3.9	-9.0
WISDM	0.2	-1.3	-4.0	0.0	1.7
Synthetic	0.2	0.5	-14.0	-11.5	2.9
NinaPro DB1	-0.1	0.0	-46.9	-46.8	0.1
	No Common	No Global	No Personal	Common Only	Personal Only

Fig. 8. Branch-structure ablation shown as relative performance change with respect to the full model.

D. Effect of HD Dimension

As shown in Fig. 7, HD dimension mainly controls how much representational capacity is available for the prototypes. In HoRU, this matters because the method must represent both the shared and the personal branches of the prototype decomposition. Some datasets remain clearly dimension-sensitive as shown in Fig. 7(a). This effect is strongest on NinaPro DB1, where the mean personalized accuracy increases monotonically from 69.7% at $d = 1000$ to 74.7% at $d = 2000$, 79.6% at $d = 5000$, and 81.7% at $d = 10000$. Synthetic also improves steadily from 72.8% to 74.2%, 75.0%, and 76.5%. It suggests that, on these datasets, the prototype representation is still short of capacity at lower dimensions, so a larger HD space remains useful.

As shown in Fig. 7(b), Other datasets are already close to saturation at moderate dimension. UCI-HAR stays nearly flat across the full range, with accuracies of 97.3%, 97.5%, 97.6%, and 97.5% for $d = 1000, 2000, 5000, 10000$, respectively. FEMNIST also changes very little, moving from 68.8% to 68.8%, 69.2%, and 69.0%. ISOLET and WISDM follow the same qualitative pattern, with only mild changes as dimension increases. For these datasets, the main prototype structure is already captured well at moderate dimension, so enlarging the HD space adds little.

Overall, these results indicate that HD dimension is not a uniformly effective performance lever, but rather depends on whether additional representational capacity is needed for a given dataset. Increasing dimension improves accuracy when prototype capacity is insufficient, but yields diminishing returns once the underlying structure is adequately captured. This behavior reflects a fundamental trade-off in HDC-based federated learning: higher dimensions increase model capacity but also directly raise communication cost. In practice, $d = 2000$ provides a strong operating point for several datasets, balancing accuracy and efficiency. Larger dimensions such as $d = 5000$ or $d = 10000$ can further improve performance on more challenging personalization tasks, such as NinaPro DB1, helping narrow the gap to higher-capacity neural network models at the expense of increased communication overhead.

E. Analysis

a) *Ablation on each branch:* To verify that the improvement of the full model comes from the proposed prototype

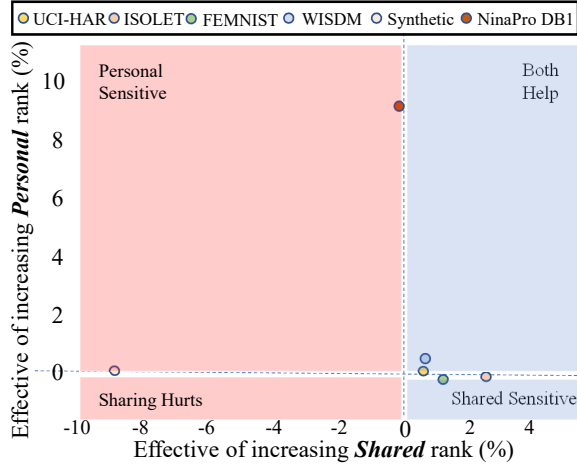


Fig. 9. Dataset-wise sensitivity to rank allocation.

decomposition rather than from low-rank modeling alone, we perform a branch-structure ablation in which one branch is removed at a time while all other training settings are kept fixed. We compare the full model with five variants: `no_common`, which removes the common branch; `no_global`, which removes the global branch; `no_personal`, which removes the personal branch; `shared_only`, which keeps only the common and global branches; and `personal_only`, which keeps only the personal branch. When one shared branch is removed, its rank budget is reassigned to the remaining shared branch so that the comparison reflects branch function rather than differences in capacity or training effort.

Fig. 8 shows that the contribution of each branch is strongly dataset-dependent. On NinaPro DB1, removing the personal branch leads to clear performance drops of 46.9%, respectively, and the `shared_only` variant degrades in a similar way. This indicates that, on these datasets, client-specific prototype information is essential and cannot be recovered well from shared branches alone.

FEMNIST shows a different pattern. Removing the common branch causes a large drop of 14.4%, whereas removing the global branch changes performance only marginally (+0.5%). This suggests that FEMNIST benefits mainly from a shareable prototype component that remains useful across clients, while a branch reserved only for strictly global prototype information contributes relatively little beyond that.

This same tendency appears more broadly across datasets. In most cases, `no_global` remains close to the full model, indicating that the marginal contribution of the global branch is limited. Although the model explicitly allocates a separate branch for purely global prototype structure, most of the useful shared signal appears to be captured already by the common branch. In other words, the dominant shareable prototype structure is usually not purely global, but lies in prototype components that are broadly shared while still remaining effective under client heterogeneity.

b) Dataset-dependent effect of rank allocation:

Fig. 9 examines how strongly each dataset depends on the capacity of the shared and personal branches in the proposed prototype decomposition. Rather than selecting

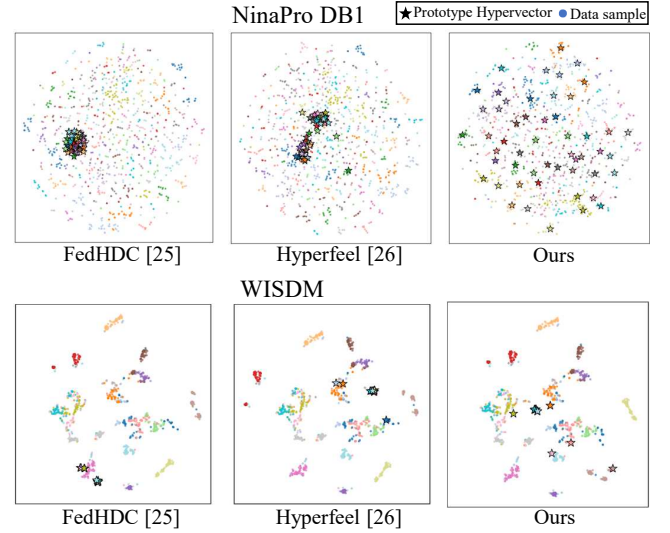


Fig. 10. t-SNE visualization of hard tail clients under natural heterogeneity.

the single best rank setting, we keep the training protocol fixed and vary only how the low-rank budget is divided across branches. The horizontal axis reports the mean accuracy change when the shared rank is increased from 16 to 64, averaged over the other sweep settings, and the vertical axis reports the analogous mean accuracy change when the personal rank is increased from 16 to 64. In this controlled sweep, we use $\text{shared_rank} \in \{16, 32, 48, 64\}$, $\text{intersection_ratio} \in \{0.25, 0.5, 0.75\}$, and $\text{personal_rank} \in \{16, 32, 48, 64\}$.

This view shows three broad dataset regimes. ISOLET is the clearest case in which performance benefits mainly from a larger shared branch: increasing the shared rank gives about +2.46%, while increasing the personal rank gives almost no gain. FEMNIST, UCI-HAR, and WISDM show the same tendency more mildly, indicating that they also rely more on additional shared prototype capacity than on additional personal capacity. In contrast, Synthetic and NinaPro DB1 are driven more by the personal branch. Synthetic shows the strongest negative response to increased shared rank, with a shared-rank delta of about -8.84%, which suggests that additional sharing is harmful. NinaPro DB1 is nearly insensitive to shared-rank expansion (-0.10%) but responds strongly to a larger personal rank (+9.15%).

This analysis shows that rank allocation can be used as a diagnostic of federated prototype structure. Datasets located farther to the right benefit more from increased shared-branch capacity, indicating stronger cross-client alignment in their prototype structure. Datasets located higher on the plot benefit more from increased personal-branch capacity, indicating that preserving client-specific structure is more important for good performance. The overall pattern therefore shows that no single shared-personal allocation is uniformly optimal across all benchmarks. Instead, the appropriate prototype decomposition depends on the extent to which each dataset is governed by shareable cross-client structure or by client-specific variability.

c) Tail-client recovery through prototype decomposition: The clearest quantitative advantage of HoRU ap-

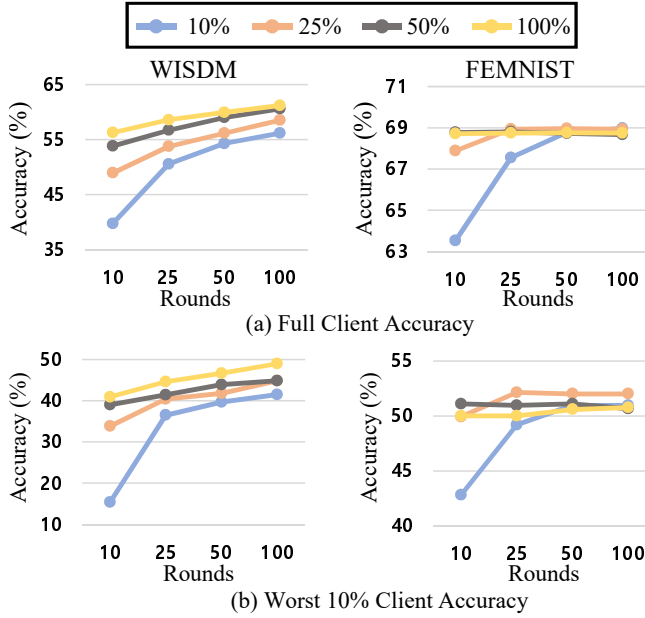


Fig. 11. Effect of client participation ratio on convergence.

pears in tail-client recovery. Under the natural split, the 10th-percentile client accuracy on WISDM is 42.9% for the full model, compared with 37.4% for *HyperFeel* and 5.4% for global *FedHDC*. On NINAPRO DB1, the corresponding values are 70.5%, 54.5%, and 23.1%. The same ordering also appears in client-average accuracy: on WISDM, *final_acc* is 57.2%, 49.6%, and 8.4%, while on NINAPRO DB1 it is 75.3%, 60.4%, and 30.5%, respectively. The worst-client metric shows a similar overall tendency, with a small exception on WISDM, where *HyperFeel* is slightly higher than HoRU on the single hardest client. Overall, these results indicate that HoRU most consistently improves *broad tail-client recovery* across low-performing clients.

Fig. 10 helps explain this result at the prototype level. In each panel, colored dots denote test samples from one hard client, and star markers denote the class prototypes produced by each method. HoRU places these prototypes closer to the corresponding client-specific sample clouds, whereas global *FedHDC* shows a larger prototype mismatch and *HyperFeel* achieves only partial recovery. This pattern supports the view that tail-client gains come from decomposing the classifier prototype into shared and personal parts: the shared part transfers what is consistent across clients, while the personal part preserves client-specific structure that would otherwise be overwritten by a single global prototype arrangement.

d) Effect of Participation Ratio on Convergence: Following the standard training setup, we keep all optimization and model settings fixed and vary only the client participation ratio in $\{10\%, 25\%, 50\%, 100\%\}$. For this study only, we extend training to 100 communication rounds. Fig. 11 shows that the participation ratio mainly affects how quickly and how smoothly HoRU converges. Because the shared bases are computed once at bootstrap using all clients, every setting starts from the same initial prototype decomposition. The difference arises during federated refinement: when fewer

clients participate in a round, the server estimates the shared prototype update from less information, which makes the round-wise update noisier; when more clients participate, the update is more stable and convergence is faster.

To quantify this effect, let ΔR_t denote the accuracy gap at round t between the 100% and 10% participation settings under the same communication-round budget. Both WISDM and FEMNIST show a clear early-round gap under sparse participation. The effect is strongest on WISDM, where $\Delta R_{10} = +16.5$ points and a non-trivial gap remains at round 100 with $\Delta R_{100} = +5.0$ points. FEMNIST shows a smaller but still visible early-round gap, with $\Delta R_{10} = +5.2$ points, while the difference is nearly zero by round 100 with $\Delta R_{100} \approx 0$. This suggests that, on FEMNIST, lower participation mainly delays convergence rather than changing the final mean accuracy.

The results in Fig. 11(b), which reports the worst 10% client accuracy, show the same pattern. On WISDM, higher participation yields a consistently stronger trajectory and a higher late-stage value, indicating faster recovery of weaker clients. On FEMNIST, the gap appears mainly in the early rounds and becomes small later. The remaining datasets follow the same trend with smaller magnitude. For example, UCI-HAR shows only $\Delta R_{10} = +3.2$ and $\Delta R_{100} = +0.5$, while several other datasets are nearly flat across participation levels.

e) Diagnostics of learned prototype structure.: To understand how the proposed decomposition is utilized by the model, we perform a controlled diagnostic study that isolates the effect of branch allocation on the learned class-hypervector structure. Within a fixed learning setup, we vary only how each class hypervector is distributed across the common, global, and personal branches. In this controlled sweep, we use $\text{shared_rank} \in \{32, 48, 64\}$, $\text{intersection_ratio} \in \{0.25, 0.75\}$, and $\text{personal_rank} \in \{32, 48, 64\}$. The shared branch is split into the common and global branches by the chosen intersection ratio, while the personal branch size is changed independently. Thus, all compared models are trained under the same optimization setting, and only the branch allocation of the class hypervector is changed.

We use two post-hoc diagnostics to read the learned prototype structure after training. Branch energy measures how much of the final prototype is carried by each branch. Let $M_{\text{common}} = CB_c^\top$, $M_{\text{global}} = GB_g^\top$, and $M_{\text{personal}} = \Delta B_c^\top + PP_i^\top$, where C , G , Δ , and P denote the learned common, global, common-delta, and personal coefficients, respectively. The branch-energy share of branch $b \in \{\text{common}, \text{global}, \text{personal}\}$ is

$$e_b = \frac{\|M_b\|_F}{\|M_{\text{common}}\|_F + \|M_{\text{global}}\|_F + \|M_{\text{personal}}\|_F}.$$

Thus, branch energy shows where the prototype mass is placed. Effective rank measures how many directions remain active in the reconstructed full class-hypervector matrix M_i^{full} , and is computed as

$$r_{\text{eff}}(M_i^{\text{full}}) = \frac{\|M_i^{\text{full}}\|_F^2}{\|M_i^{\text{full}}\|_2^2}.$$

A larger effective rank indicates that the learned prototype uses a richer set of directions, whereas a value near 1 indicates

that the class hypervectors space is concentrated in only a few directions.

In FEMNIST, the learned class-hypervector representation places most of its energy in the common branch. Across all allocations, the common-energy share stays high at 0.884–0.911, the personal-energy share stays low at 0.084–0.094, the global-energy share remains very small at 0.004–0.023, and the effective rank stays near 1.00. The best accuracy also appears when the shared branch allocates more capacity to the common part and the personal rank remains small. This indicates that FEMNIST stores most useful prototype information in a strongly shared common branch, while additional personal capacity contributes little.

In WISDM, the learned class-hypervector representation distributes its energy across both the shared and personal branches. For the stronger settings, the common branch carries about 0.444–0.471 of the energy, the global branch about 0.091–0.114, and the personal branch about 0.438–0.442. At the same time, the effective rank rises from 3.03 to 3.52. This indicates that WISDM benefits when the prototype uses both a larger common branch and a substantial personal branch, rather than concentrating most of the prototype information in only one branch.

In NinaPro DB1, the learned class-hypervector representation places most of its energy in the personal branch. The personal-energy share is the largest component in every setting at 0.732–0.806, while the common-energy share remains much smaller at 0.131–0.194 and the global-energy share stays at 0.039–0.103. The effective rank also reaches 27.78, much higher than in FEMNIST and WISDM. Accuracy follows the same direction, increasing as the personal rank becomes larger. This indicates that NinaPro DB1 stores most useful prototype information in a rich client-specific personal branch rather than in the shared branches.

Across these datasets, the global branch does not carry much of the useful prototype information. Instead, most shareable information is placed in the common branch, while the personal branch preserves client-specific information. This suggests that the global branch mainly serves as a stable shared reference during training, rather than as the main source of performance gain.

V. RELATED WORK

A. Federated Hyperdimensional Computing

Hyperdimensional computing (HDC) has recently emerged as a practical direction for federated learning under edge constraints because it replaces heavy gradient-based training with high-dimensional encoding, prototype-style learning, and simple vector operations [29], [30]. Early work such as FL-HDC [31] showed that HDC can be integrated into the federated setting as a compact and hardware-friendly framework. More recent methods, including **FedHDC** [6] and **HyperFeel** [12], continue this direction by emphasizing lightweight local updates, prototype-based model exchange, and reduced communication cost. Resource-efficient federated HDC variants further reinforce this trend toward communication- and memory-conscious deployment [23].

However, most existing federated HDC methods focus primarily on efficiency and lightweight federation. Under heterogeneous clients, the remaining issue is not only how to communicate less, but also how to decide what part of the prototype memory should be shared. Prior methods still largely treat prototype memory as a single shared object [6], [12], which can weaken alignment to client-local structure when client distributions differ substantially. Our work addresses this gap by decomposing prototype memory into synchronized shared structure and retained personal structure, rather than federating one holistic prototype space.

B. Personalized and Disentangled Federated Learning

A broader line of federated learning research addresses heterogeneity through personalization. Representative directions include personalization layers [32], multi-task formulations [33], meta-learning-based personalization [34], and regularized personalized optimization such as pFedMe [35]. Methods such as Ditto [36] further show that personalized local models can improve robustness and client-level performance when a single global model is not sufficient.

A closely related direction is disentangled federated learning. DFL [28] makes the shared/private distinction explicit by separating globally useful information from client-specific structure through disentangled branches. This is especially relevant to our setting because it frames heterogeneity not as a minor refinement, but as a structural problem of separating what should be aggregated from what should remain local.

The main difference is that these personalized and disentangled FL methods are mostly developed for neural networks-based backbones, personalized heads, or feature-level branch separation [28], [35], [36]. In HDC, however, the main learning object is not a layered feature hierarchy but a class prototype memory distributed across high-dimensional coordinates. Our work brings the shared–private principle into this prototype-centric HDC setting by decomposing the memory itself into components with different federated roles.

VI. CONCLUSION

We presented HoRU, a personalized federated hyperdimensional computing framework that replaces holistic prototype sharing with a shared–personal memory decomposition. The key idea is to treat client prototype memory not as a single object to be either fully shared or fully localized, but as a composition of common shared, global-only, and personal structure. This enables clients to preserve local geometry while still benefiting from cross-client regularization through low-rank shared synchronization.

Across seven federated benchmarks, HoRU improved mean accuracy by +4.3% points over HyperFeel and +21.2% points over FedHDC. Beyond accuracy, the method also delivered clear systems benefits on a reduced synthetic benchmark, making client rounds about $3.0\times$ faster than prior federated HDC baselines while reducing uploaded payload by $62.5\times$. These results show that personalization in federated HDC should be expressed directly at the prototype-memory level, rather than borrowed indirectly from neural multi-branch designs.

REFERENCES

- [1] R. Singh and S. S. Gill, "Edge ai: A survey," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2667345223000196>
- [2] H. Kwon, K. Kim, J. Lee, H. Lee, J. Kim, J. Kim, T. Kim, Y. Kim, Y. Ni, M. Imani, I. Suh, and Y. Kim, "Brain-inspired hyperdimensional computing in the wild: Lightweight symbolic learning for sensorimotor controls of wheeled robots," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 5176–5182.
- [3] J. Kim, H. Lee, M. Imani, and Y. Kim, "Efficient hyperdimensional learning with trainable, quantizable, and holistic data representation," in *2023 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2023, pp. 1–6.
- [4] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L. D'Oliveira, H. Eichner, S. E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, M. Raykova, H. Qi, D. Ramage, R. Raskar, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, and S. Zhao, "Advances and open problems in federated learning," 2021. [Online]. Available: <https://arxiv.org/abs/1912.04977>
- [5] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," 2023. [Online]. Available: <https://arxiv.org/abs/1602.05629>
- [6] K. Ergun, R. Chandrasekaran, and T. Rosing, "Federated hyperdimensional computing," 2023. [Online]. Available: <https://arxiv.org/abs/2312.15966>
- [7] Q. Zhao, K. Lee, J. Liu, M. Huzaifa, X. Yu, and T. Rosing, "Fedhd: federated learning with hyperdimensional computing," in *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*, ser. MobiCom '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 791–793. [Online]. Available: <https://doi.org/10.1145/3495243.3558757>
- [8] H. Lee, W. Han, H. Kim, H. Kwon, S. Jang, I. Suh, and Y. Kim, "Hyperdimensional computing-based federated learning in mobile robots through synthetic oversampling," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 13 406–13 412.
- [9] C.-Y. Hsieh, Y.-C. Chuang, and A.-Y. A. Wu, "Fl-hdc: Hyperdimensional computing design for the application of federated learning," in *2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 2021, pp. 1–5.
- [10] M. Atzori, A. Gijsberts, I. Kuzborskij, S. Heynen, A.-G. M. Hager, O. Deriaz, C. Castellini, H. Müller, and B. Caputo, "Characterization of a benchmark database for myoelectric movement classification," *Transactions on Neural Systems and Rehabilitation Engineering*, vol. 23, no. 1, pp. 73–83, Jan. 2015, (In press). [Online]. Available: <http://ieeexplore.ieee.org/xpl/articleDetails.jsp?arnumber=6825822>
- [11] G. Weiss, "WISDM Smartphone and Smartwatch Activity and Biometrics Dataset," UCI Machine Learning Repository, 2019, DOI: <https://doi.org/10.24432/C5HK59>.
- [12] H. Li, F. Liu, Y. Chen, and L. Jiang, "Hyperfeel: An efficient federated learning framework using hyperdimensional computing," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024, pp. 716–721.
- [13] A. Rahimi, S. Datta, D. Kleyko, E. P. Frady, B. Olshausen, P. Kanerva, and J. M. Rabaey, "High-dimensional computing as a nanoscale paradigm," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 64, no. 9, pp. 2508–2521, 2017.
- [14] M. Stock, D. Boeckkaerts, P. Dewulf, S. Taelman, M. V. Haeverbeke, W. V. Crikinge, and B. D. Baets, "Hyperdimensional computing: A fast, robust, and interpretable paradigm for biological data," *PLOS Computational Biology*, vol. 20, no. 9, p. e1012426, 2024.
- [15] H. Lee, J. Kim, S. Kim, H. Kwon, M. Imani, I. Suh, and Y. Kim, "Efficient forward-only training for brain-inspired hyperdimensional computing," in *2024 IEEE 42nd International Conference on Computer Design (ICCD)*. IEEE, 2024, pp. 707–714.
- [16] T. Yu, Y. Zhang, Z. Zhang, and C. D. Sa, "Understanding hyperdimensional computing for parallel single-pass learning," 2023. [Online]. Available: <https://arxiv.org/abs/2202.04805>
- [17] A. Rahimi, P. Kanerva, and J. M. Rabaey, "A robust and energy-efficient classifier using brain-inspired hyperdimensional computing," in *Proceedings of the International Symposium on Low Power Electronics and Design (ISLPED)*, 2016, pp. 64–69.
- [18] Z. Chen, D. Hoang, F. J. Piran, R. Chen, and F. Imani, "Federated hyperdimensional computing for hierarchical and distributed quality monitoring in smart manufacturing," *Internet of Things*, vol. 31, p. 101568, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2542660525000812>
- [19] Q. Zhao, X. Yu, S. Hu, and T. Rosing, "Multimodalhd: Federated learning over heterogeneous sensor modalities using hyperdimensional computing," in *2024 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2024, pp. 1–6.
- [20] Y. Tian, R. Chandrasekaran, K. Ergun, X. Yu, and T. Rosing, "Federated hyperdimensional computing: Comprehensive analysis and robust communication," *ACM Trans. Internet Things*, vol. 6, no. 3, May 2025. [Online]. Available: <https://doi.org/10.1145/3724129>
- [21] H. Lee, W. Han, H. Kim, H. Kwon, S. Jang, I. Suh, and Y. Kim, "Hyperdimensional computing-based federated learning in mobile robots through synthetic oversampling," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 13 406–13 412.
- [22] H. Kwon, K. Kim, J. Lee, H. Lee, J. Kim, J. Kim, T. Kim, Y. Kim, Y. Ni, M. Imani *et al.*, "Brain-inspired hyperdimensional computing in the wild: Lightweight symbolic learning for sensorimotor controls of wheeled robots," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 5176–5182.
- [23] N. Zeulin, O. Galina, N. Himayat, and S. Andreev, "Resource-efficient federated hyperdimensional computing," 2023. [Online]. Available: <https://arxiv.org/abs/2306.01339>
- [24] M. Imani, S. Pampana, S. Gupta, M. Zhou, Y. Kim, and T. Rosing, "Dual: Acceleration of clustering algorithms using digital-based processing in-memory," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 356–371.
- [25] A. D. G. A. O. L. Reyes-Ortiz, Jorge and X. Parra, "Human Activity Recognition Using Smartphones," UCI Machine Learning Repository, 2013, DOI: <https://doi.org/10.24432/C54S4K>.
- [26] R. Cole and M. Fanty, "ISOLET," UCI Machine Learning Repository, 1991, DOI: <https://doi.org/10.24432/C51G69>.
- [27] S. Caldas, S. M. K. Duddu, P. Wu, T. Li, J. Konečný, H. B. McMahan, V. Smith, and A. Talwalkar, "Leaf: A benchmark for federated settings," 2019. [Online]. Available: <https://arxiv.org/abs/1812.01097>
- [28] Z. Luo, Y. Wang, Z. Wang, Z. Sun, and T. Tan, "Disentangled federated learning for tackling attributes skew via invariant aggregation and diversity transferring," 2022. [Online]. Available: <https://arxiv.org/abs/2206.06818>
- [29] D. Kleyko, D. Rachkovskij, E. Osipov, and A. Rahimi, "A survey on hyperdimensional computing aka vector symbolic architectures, part ii: Applications, cognitive models, and challenges," *ACM Computing Surveys*, vol. 55, no. 9, p. 1–52, Jan. 2023. [Online]. Available: <http://dx.doi.org/10.1145/3558000>
- [30] M. Heddes, I. Nunes, T. Givargis, A. Nicolau, and A. Veidenbaum, "Hyperdimensional computing: a framework for stochastic computation and symbolic ai," *Journal of Big Data*, vol. 11, no. 1, p. 145, 2024.
- [31] C.-Y. Hsieh, Y.-C. Chuang, and A.-Y. A. Wu, "Fl-hdc: Hyperdimensional computing design for the application of federated learning," in *2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, 2021, pp. 1–5.
- [32] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated learning with personalization layers," 2019. [Online]. Available: <https://arxiv.org/abs/1912.00818>
- [33] V. Smith, C.-K. Chiang, M. Sanjabi, and A. Talwalkar, "Federated multi-task learning," 2018. [Online]. Available: <https://arxiv.org/abs/1705.10467>
- [34] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach," in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 3557–3568. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/24389bfe4fe2eba8bf9aa9203a44cdad-Paper.pdf
- [35] C. T. Dinh, N. H. Tran, and T. D. Nguyen, "Personalized federated learning with moreau envelopes," 2022. [Online]. Available: <https://arxiv.org/abs/2006.08848>
- [36] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," 2021. [Online]. Available: <https://arxiv.org/abs/2012.04221>